

1) Functional Requirements

Core (MVP)

- **User accounts & identity**
 - Sign up / login (email/phone/OAuth), profile, presence (online/offline/last seen).
- **1:1 messaging**
 - Send/receive text messages, delivery states (**sent / delivered / read**).
 - Message ordering per conversation, retries, deduplication.
- **Group chats**
 - Create group, add/remove members, roles (owner/admin/member).
- **Conversation list**
 - List user's chats with last message preview, unread counts.
- **Message history**
 - Pagination (by time/message id), jump to newest, search basic (optional).
- **Attachments (optional for MVP, common in interviews)**
 - Send images/files, get thumbnails, download.
- **Push/Web notifications**
 - Notify user when offline / background.
- **Typing indicator + receipts** (can be later, but common)
- **Basic moderation**
 - Block user, report abuse.

Nice-to-have (later)

- Message edit/delete, reactions, mentions, threads, voice notes, call integration.
- Multi-device sync, "message delivered to device(s)" semantics.
- End-to-end encryption, disappearing messages.

2) Non-Functional Requirements

Scale & performance

- **Low latency:** p95 send→deliver < ~200ms (regional), p99 < ~1s.
- **High throughput:** handle spikes (celebrity group chat, events).
- **Availability:** 99.9%+ for messaging APIs; graceful degradation.
- **Consistency targets:**
 - **Per-conversation ordering** (strong-ish within a chat).
 - **Eventual consistency** for presence, read receipts.

Reliability

- **At-least-once delivery** between services; client-level dedup needed.
- Durable storage for message history.
- Idempotent APIs and retry-safe consumers.

Security & compliance

- TLS everywhere, encryption at rest, access control, audit logging.
- PII minimization, retention policies, GDPR delete/export (if needed).

Operability

- Monitoring (latency, queue depth, fanout failures), tracing, alerting.
- Rate limiting + abuse detection.

3) Back-of-the-envelope Estimation (example numbers)

Assume:

- **DAU:** 10M
- **Avg messages/user/day:** 50
→ total messages/day = $10M * 50 = 500M \text{ msgs/day}$
- **Average message payload (text + metadata):** ~1 KB stored (rough)
- **Write QPS:**
 $500M / 86,400 \approx 5,787 \text{ msg/s}$ average
Peak x5 → ~30K msg/s
- **Storage/day (messages only):**
 $500M * 1 \text{ KB} = 500 \text{ GB/day}$
- **Storage/year:**
 $500 \text{ GB/day} * 365 \approx 182.5 \text{ TB/year}$ (before replication/indexing)
- **Fanout:**
1:1 average recipients ~1, groups vary. If average recipients = 1.2, then delivery events ~ **600M/day**.
- **Realtime connections:**
If 20% concurrently online from DAU: 2M concurrent WebSocket connections (big; sharding gateways required).

These numbers justify:

- Partitioned storage, queue-based delivery, horizontally scaled WS gateways, cache for inbox.
-

4) High-Level Design

A) API Design (external)

Auth

- `POST /v1/auth/signup`
- `POST /v1/auth/login`
- `POST /v1/auth/refresh`

Conversations

- `POST /v1/conversations` (create 1:1 or group)
- `GET /v1/conversations?cursor=...`
- `GET /v1/conversations/{cid}`
- `POST /v1/conversations/{cid}/members` (add)
- `DELETE /v1/conversations/{cid}/members/{uid}` (remove)

Messages

- `POST /v1/conversations/{cid}/messages`
- `GET /v1/conversations/{cid}/messages?before=...&limit=...`
- `POST /v1/conversations/{cid}/read` (read up to msg_id/time)
- `POST /v1/conversations/{cid}/typing` (ephemeral)

Realtime (WebSocket)

- `GET /v1/ws` (upgrade)
 - Client sends auth token, subscribes to conversation channels.

Attachments

- `POST /v1/attachments/presign` (get upload URL)
- `POST /v1/messages/{mid}/attachments` (attach metadata)

B) Contract Design (service-to-service events)

Use a message bus (Kafka/PubSub). Define versioned events:

MessageCreated v1

```
{
  "event_id": "uuid",
  "type": "MessageCreated",
  "version": 1,
  "message_id": "uuid",
  "conversation_id": "cid",
  "sender_id": "uid",
  "client_msg_id": "string-for-dedup",
  "sequence": 123456,
  "timestamp_ms": 1730000000000,
  "content": { "kind": "text", "text": "hi" }
}
```

DeliveryRequested / Delivered / Read

- Delivered is per recipient device/user.
- Read is typically “user read up to sequence”.

PresenceUpdated (ephemeral)

- Best-effort, not persisted long-term.

Idempotency

- `event_id` + `client_msg_id` used for dedup.
-

C) Data Model

Option 1: “Inbox + Message store” (common at scale)

Messages table (append-only)

- Partition key: `conversation_id`
- Sort key: `sequence` (monotonic per conversation)
- Columns: `sender_id`, `timestamp`, `content`, `attachments`
- Storage: Cassandra/DynamoDB (wide-column / KV with range queries)

ConversationMembers

- (`conversation_id`, `user_id`) → `role`, `joined_at`, `muted`, `last_read_seq`

UserInbox

- Partition key: `user_id`
- Sort key: `last_activity_ts` or `conversation_id`
- Columns: `last_message_seq`, `unread_count`, `pinned`, etc.
- This is what powers conversation list quickly.

Dedup table

- (`sender_id`, `client_msg_id`) → `message_id`, `created_at` (TTL)

Attachments metadata

- message_id → blob keys, sizes, mime, thumbnail key

Why this model works

- Message history is naturally by conversation and paginated by sequence/time.
 - Inbox is per-user and fast to render.
 - Read state lives per (conversation, user).
-

D) Architectural Design

Components

1. API Gateway

- Auth, rate limiting, routing.

2. Auth Service

- Token issuance (JWT/OAuth), refresh, device sessions.

3. Chat Service

- Conversation management, membership, permissions.

4. Message Service

- Validates send, assigns sequence, writes message store.

5. Fanout/Delivery Service

- Determines recipients, writes to UserInbox/unread counters, emits delivery events.

6. Realtime Gateway (WebSocket fleet)

- Maintains connections; subscribes to delivery events; pushes to clients.

7. Notification Service

- Push notifications (APNs/FCM) when not connected.

8. Media Service

- Presigned upload, virus scan, thumbnail generation.

Send message flow

1. Client → `POST /messages` with `client_msg_id` (idempotency key).
2. Message Service:
 - AuthZ: user is member of conversation.
 - Assign **sequence** (via per-conversation sequencer; options below).
 - Persist message.
 - Publish `MessageCreated`.
3. Fanout Service consumes:
 - Looks up conversation members.

- Updates each recipient's inbox/unread.
 - Publishes `DeliveryRequested` or directly `Delivered` events.
4. Realtime Gateway pushes to connected recipients.
 5. Recipients ack delivery/read; service updates states.

Sequencing options (pick one)

- **Per-conversation leader/partition:** all messages for a conversation routed to one partition/worker → assigns incrementing sequence (simple, scalable).
- **Database conditional write:** store "last_seq" and CAS increment (can be hot).
- **Snowflake/time-based ordering** is not enough if you need strict per-chat ordering.

Realtime delivery

- WS gateways are stateless; store connection registry in Redis (uid → gateway instance).
- If gateway dies: reconnect and resubscribe; missed messages fetched via history API.

Caching

- Redis for:
 - membership cache (conversation_id → members)
 - inbox hot data
 - presence/typing ephemeral

E) Security & Privacy

Transport & storage

- TLS for all client/server and service/service.
- Encrypt at rest (KMS-managed keys).
- Secure object storage for attachments with short-lived signed URLs.

AuthZ

- Every message send/read checks membership.
- Group role-based permissions (admin actions).
- Device session management and token rotation.

Abuse prevention

- Rate limit sends, attachment uploads.
- Spam detection heuristics (link frequency, bursts).
- Block lists enforced in fanout.

Privacy

- Data minimization: store only necessary metadata.
- Configurable retention (e.g., delete after N days for certain chats).

- GDPR-style delete/export hooks (delete user → tombstone conversations, redact messages where required by policy).

E2EE (optional advanced)

- If required: server stores only ciphertext, clients manage keys; receipts/typing become more complex.
-

If you want, I can also provide:

- A concrete choice of storage (e.g., Cassandra + Redis + Kafka) with partition keys and example schemas,
- A sequence assignment strategy in more detail,
- Failure modes (fanout lag, duplicate deliveries) and how to handle them.